

Source:

frontend/src/components/atoms/Text.jsx

[Home](#)

[Namespaces](#)

[apiConfig](#)

[contactoEmergenciaService](#)

[diarioService](#)

[Classes](#)

[ErrorBoundary](#)

[Events](#)

[SIGINT](#)

[unhandledRejection](#)

[Global](#)

[404_Error_Handler](#)

[ACTIVIDADES](#)

[API_URL](#)

[ASPECTOS_COGNITIVOS](#)

[ActionCard](#)

[ActivitySelector](#)

[App](#)

[AuthButtons](#)

[AuthLayout](#)

[Button](#)

[CORS_Configuration](#)

[Card](#)

[Carousel](#)

[Checkbox](#)

[CircularProgress](#)

[CognitionSelector](#)

[DiaryBanner](#)

[DiaryEditor](#)

[DiaryEntry](#)

[EMOCIONES](#)

[EmergencyButton](#)

[EmergencyModal](#)

[EmotionSelector](#)

[EmotionStats](#)

[Footer](#)

[GET_/](#)

[Generic_Error_Handler](#)

[Heading](#)

[HeroSection](#)

[Home](#)

[Input](#)

[Landing](#)

[LandingHero](#)

[Login](#)

[Logo](#)

[MainLayout](#)

[Navbar](#)

[ProtectedRoute](#)

[Register](#)

[Seguimiento](#)

[Slider](#)

```
1.  /**
2.   * @file Componente de texto reutilizable.
3.   * @description Renderiza un elemento de párrafo con diferente
4.   * @requires react
5.   * @requires prop-types
6.   * @requires ../../styles/atoms/Text.css
7.   */
8.  import React from "react";
9.  import PropTypes from "prop-types";
10. import "../../styles/atoms/Text.css";
11.
12. /**
13.  * @function Text
14.  * @description Renderiza un componente de texto con variantes
15.  * @param {object} props - Las propiedades del componente.
16.  * @param {React.ReactNode} props.children - El contenido del
17.  * @param {string} [props.variant='body'] - La variante de est
18.  * @param {string} [props.className=''] - Clases CSS adicional
19.  * @returns {JSX.Element} El componente de texto.
20.  */
21. const Text = ({
22.   children,
23.   variant = 'body',
24.   className = ''
25. }) => {
26.   const classNames = ['text', `text--${variant}`, className]
27.     .filter(Boolean)
28.     .join(' ');
29.
30.   return (
31.     <p className={classNames}>
32.       {children}
33.     </p>
34.   );
35. };
36.
37. Text.propTypes = {
38.   /** El contenido del texto. */
39.   children: PropTypes.node.isRequired,
40.   /** La variante de estilo del texto. */
41.   variant: PropTypes.oneOf(['body', 'subtitle', 'caption', '
42.   /** Clases CSS adicionales para aplicar al texto. */
```

```
43.         className: PropTypes.string
44.     };
45.
46. export default Text;
```

Text
Textarea
accessWithPassword
actualizarEntradaDiario
api
apiFetch
authMiddleware
closeShareModal
compararPassword
componentDidCatch
connectDBAndStartServer
copyShareLink
crearEntradaDiario
create
createContacto
createRegistro
delete
deleteContacto
description
dismiss
duracion
eliminarEntradaDiario
emotion
error
formatDate
formatDateShort
getAll
getById
getContactoById
getContactos
getDerivedStateFromError
getIntensidad
getPreviewText
getProfile
getRegistroByFecha
getRegistroById
getRegistros
getRegistrosByRango
getStepLabel
goToNext
goToPrevious
handleAddActividad
handleAddContact
handleCall
handleCancelAdd
handleCancelEditor
handleChange
handleCreateEntry
handleDeleteEntry
handleEditEntry
handleEmocionToggle
handleInputChange
handleIntensidadChange
handleLogin
handleLogout
handleOverlayClick
handleRegister
handleReload
handleRemoveActividad

handleSendEmail
handleSendSms
handleShareEntry
handleSubmit
handleToggle
handleToggleExpand
handleUpdateEntry
info
intensidad
isMobileDevice
isSelected
loadContacts
loadEntries
loading
loginUser
nextStep
nombre
obtenerEntradaDiarioPorId
obtenerEntradasDiario
optionalAuthMiddleware
percentage
port
pre-save
prevStep
promise
registerUser
render
request-interceptor
response-interceptor
sendEmergencyEmail
success
timeAgo
title
togglePasswordField
update
updateContacto
uri
useAuthStore
useToast
validateRequiredEnvVars